

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки



Лабораторна робота
З дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав: ст.гр IX-22

Горячий Ростислав

Прийняв:

Гордійчук-Бублівська О.В

Львів – 2026

Тема:

Реалізація клієнт-серверної взаємодії на основі протоколу WebSocket

Мета:

Розробити клієнт-серверну архітектуру, ознайомитися з WebSocket протоколом.

Розробити WebSocket-клієнт `WebSocketClient`, який дозволяє підключатися до сервера, надсилати повідомлення та отримувати відповіді в асинхронному режимі.

Вимоги до виконання:

Реалізувати клас `WebSocketClient`, що містить такі асинхронні методи:

`connect(url)`: встановлює WebSocket-з'єднання за вказаною адресою.

`send_message(message)`: надсилає повідомлення серверу.

`receive_message()`: отримує повідомлення від сервера.

`close_connection()`: закриває WebSocket-з'єднання.

Використовувати реальний WebSocket-сервер для тестування.

Реалізувати логіку обробки помилок, якщо сервер не відповідає або з'єднання перервано.

Надати приклад використання класу `WebSocketClient`, що демонструє відправку та отримання повідомлень в асинхронному режимі.

```
et.py > ...
1  import websockets
2  import asyncio
3
4  uri="wss://ws.postman-echo.com/raw"
5  class Websocket:
6      def __init__(self, uri: str):
7          self.uri=uri
8          self.ws=None
9      async def connect(self):
10         self.ws= await websockets.connect(self.uri)
11         await self.ws.send("qwerty")
12         res=await self.ws.recv()
13         print(f"відповідь сервера: {res}")
14     async def send(self,info):
15         await self.ws.send(info)
16     async def rx(self):
17         res=await self.ws.recv()
18         print(f"прийнято: {res}")
19     async def close(self):
20         await self.ws.close()
21         print("connection lost")
22 msg="HL"
23 async def main():
24     websocket = Websocket(uri)
25     try:
26         await websocket.connect()
27         await websocket.send(msg)
28         await websocket.rx()
29     except Exception as e:
30         print(f"error: {e}")
31     finally:
32         if websocket.ws:
33             await websocket.close()
34     asyncio.run(main())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
відповідь сервера: qwerty
прийнято: HL
connection lost
PS C:\Users\w1olf\Downloads\dwd>
```

Висновок:

У межах лабораторної роботи я розробив клієнт-серверну архітектуру та ознайомився з принципами роботи протоколу **WebSocket**. Отримані знання дозволили зрозуміти механізми забезпечення двостороннього зв'язку в реальному часі між клієнтом та сервером